



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: Rosas Martinez Francisco Javier

N° de Cuenta: 320109766

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 27/Septiembre/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1:

Importar su modelo de coche propio dentro del escenario a una escala adecuada.

Para resolver el ejercicio, únicamente importé el modelo del coche previamente modificado en Blender, al cual le había quitado las llantas y el cofre. Luego lo adapté a una escala adecuada tomando como referencia el modelo de Goddard.

Bloques de código generado

```
Model coche;
```

```
coche = Model();  
coche.LoadModel("Models/carro2.fbx");
```

```
// Coche base  
model = glm::mat4(1.0);  
model = glm::scale(model, glm::vec3(7.0f, 7.0f, 7.0f));  
model = glm::translate(model, glm::vec3(0.0f, 0.8f, 0.0f));  
model = glm::translate(model, glm::vec3(mainWindow.getarticulacion3(), 0.0f, 0.0f));  
model = glm::translate(model, glm::vec3(mainWindow.getarticulacion4(), 0.0f, 0.0f));  
modelaux = model;  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
color = glm::vec3(0.2f, 0.2f, 0.2f);  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
coche.RenderModel();
```

Código en ejecución



Ejercicio 2:

Importar sus 4 llantas y acomodarlas jerárquicamente, agregar el mismo valor de rotación a las llantas para que al presionar puedan rotar hacia adelante y hacia atrás.

En este ejercicio primero separé una llanta del coche en Blender y luego la importé varias veces, una por cada llanta del modelo. Después le cambié el color y le asigné dos teclas para que, al presionarlas, las llantas giraran en ambos sentidos.

Bloques de código generado

```
Model llanta;
```

```
llanta = Model();  
llanta.LoadModel("Models/llanta3.fbx");
```

```
// Llanta 1  
model = modelaux;  
model = glm::translate(model, glm::vec3(-1.91f, -0.7f, -0.9f));  
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));  
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, -1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
color = glm::vec3(0.0f, 1.0f, 0.2f);  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
llanta.RenderModel();  
  
// Llanta 2  
model = modelaux;  
model = glm::translate(model, glm::vec3( 1.30f, -0.7f, -0.9f));  
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));  
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, -1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
color = glm::vec3(0.0f, 1.0f, 0.2f);  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
llanta.RenderModel();  
  
// Llanta 3  
model = modelaux;  
model = glm::translate(model, glm::vec3(-1.91f, -0.7f, 0.9f));  
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));  
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, -1.0f));  
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
color = glm::vec3(0.0f, 1.0f, 0.2f);  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
llanta.RenderModel();  
  
// Llanta 4  
model = modelaux;  
model = glm::translate(model, glm::vec3(1.30f, -0.7f, 0.9f));  
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));  
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, -1.0f));  
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
color = glm::vec3(0.0f, 1.0f, 0.2f);  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
llanta.RenderModel();
```

Código en ejecución



Ejercicio 3:

Importar el cofre del coche, acomodarlo jerárquicamente y agregar la rotación para poder abrir y cerrar.

En este ejercicio también tuve que separar manualmente el cofre del coche utilizando Blender, para después poder instanciarlo y asignarle dos teclas que permitieran abrirlo y cerrarlo, simulando el movimiento real de un cofre.

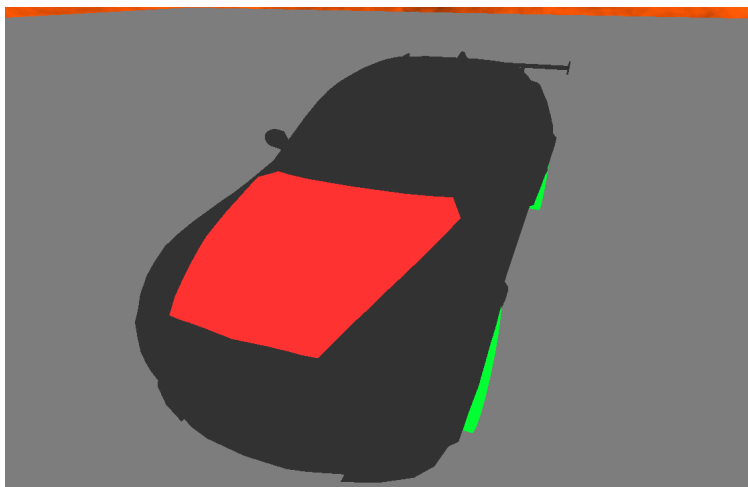
Bloques de código generado

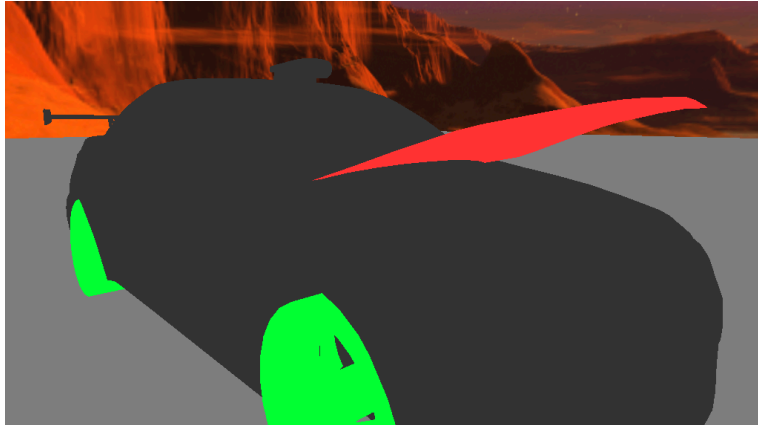
```
Model cofre;
```

```
cofre = Model();  
cofre.LoadModel("Models/cofre.fbx");
```

```
// Cofre  
model = modelaux;  
model = glm::translate(model, glm::vec3(-1.5f, 0.1f, 0.0f));  
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, -1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
color = glm::vec3(1.0f, 0.2f, 0.2f);  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
cofre.RenderModel();
```

Código en ejecución





Ejercicio 4:

Agregar traslación con teclado para que pueda avanzar y retroceder de forma independiente.

Lo que hice fue que al presionar una tecla cambie el valor de `getarticulacion4` o `getarticulacion5`, y esas dos líneas usan esos valores para empujar al objeto sobre el eje X. Como la matriz `model` va acumulando los cambios, se siente que el objeto realmente se va moviendo cada vez que aprieto una tecla, asimilando movimiento en una dirección.

Bloques de código generado

```
model = glm::translate(model, glm::vec3(mainWindow.getarticulacion4(), 0.0f, 0.0f));  
model = glm::translate(model, glm::vec3(mainWindow.getarticulacion5(), 0.0f, 0.0f));
```

Código en ejecución



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

- Con esta práctica afortunadamente no tuve ningún inconveniente.

Conclusión:

Al finalizar esta práctica pude reforzar mis conocimientos sobre la jerarquía al instanciar objetos. También aprendí a importar modelos en distintos formatos, como .obj y .fbx. Con el coche que había elegido previamente logré separar sus distintas partes, de manera que las llantas pudieran girar, el cofre abrirse y todo el vehículo desplazarse en una dirección al presionar una tecla. Fue una práctica muy entretenida que me permitió poner en práctica lo aprendido en clase de teoría y comprobar cómo esos conceptos se aplican en un proyecto real.

Bibliografía :

OpenGL Architecture Review Board, Shreiner, D., Woo, M., Neider, J., & Davis, T. (2010). OpenGL Programming Guide: The Official Guide to Learning OpenGL (7th ed.). Addison-Wesley.

OpenGL. (s. f.). OpenGL Documentation. Khronos Group. Recuperado de <https://www.khronos.org/opengl/>

Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3 (8th ed.). Addison-Wesley Professional.